ChatGPT каза:

Анализ на плановете на PNP: трябва ли да се разделят?

Да. След прегледа на шестте плана смятам, че PNP трябва да премине към разделяне на плановете на самостоятелни артефакти. Но не просто към по-малки Markdown файлове.

Моята препоръка е да се разделят три различни неща, които сега съжителстват в един документ:

1. **Какво трябва да се построи** — одобрените изисквания, решения и изпълними тикети.
2. **Как е взето решението** — readiness цикъл, одиторски блокери, ревизии и операторски арбитражи.
3. **Какво действително е изпълнено** — тестове, кодови комити, одиторски присъди, корекционни рундове и completion records.

Това е архитектурна промяна в начина, по който PNP управлява информацията. **Не предлагам да се отслабят readiness процесът, одитите или проследимостта. Предлагам агентите да получават само информацията, необходима за текущата им работа.**

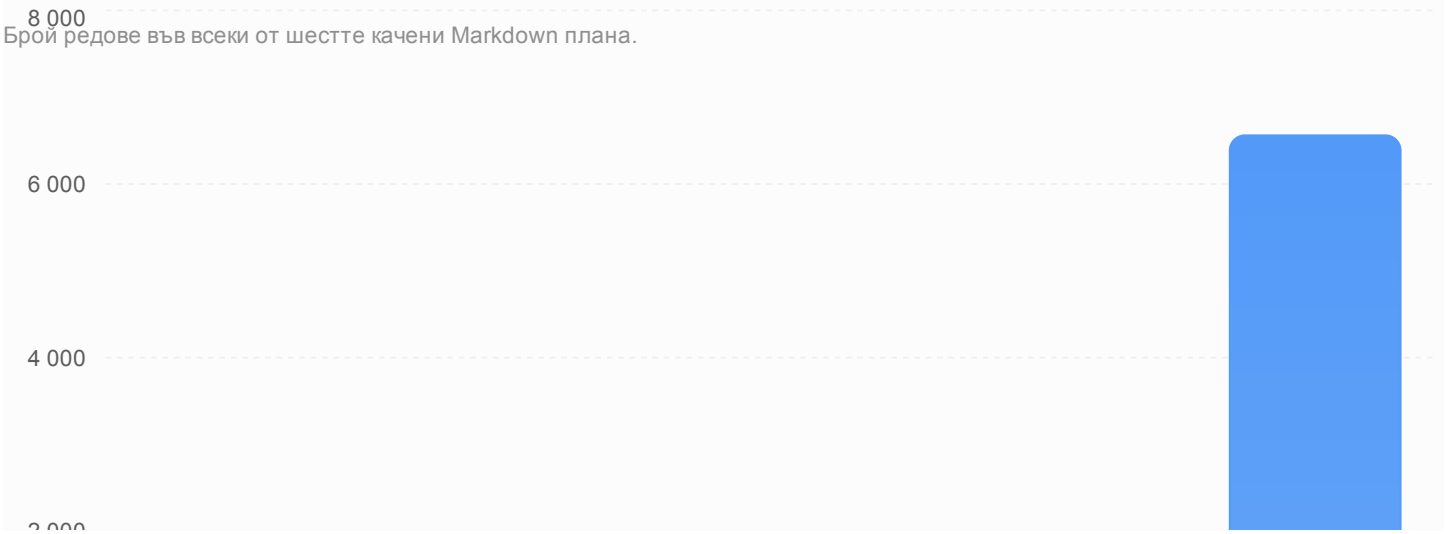
1. Какво показват шестте реални плана

Измерих съдържанието на качените файлове, а не само началните им секции.

План	Редове	Размер	Състояние
LIV	1 088	114 KB	Архивиран
FIX	607	45 KB	Архивиран
TRF	1 635	168 KB	Архивиран
PUR	1 545	197 KB	Архивиран
DPU	1 008	107 KB	Архивиран
UIR	6 561	883 KB	Активен

Размерите са приблизително в десетични KB от UTF-8 текста. Това не са измерени Claude токени.

Брой редове във всеки от шестте качени Markdown плана.





PLAN_UIR е приблизително пет пъти по-голям от средния обем на петте архивирани плана.

Самият размер обаче не е главният проблем. **Проблемът е, че различните етапи на работата изискват различни части от този размер.**

Например Writer за `UIR-050` няма нужда от пълната история на `UIR-001`, readiness блокерите на Ф2 или тестовите резултати на `UIR-038`. Но трябва да получи всички актуални архитектурни правила и зависимости, които засягат неговия тикет.

Това разграничение е ключово.

2. Къде точно се натрупва излишният контекст

Нека разгледаме анатомията на плановете.

Проблем А: изпълнимите тикети съдържат и историята на своето изпълнение

При петте архивирани плана измерих колко реда има от първия `Completion record` на всеки тикет до началото на следващия раздел.

План	Редове с completion записи*	Дял от файла
LIV	173	15,9%
FIX	161	26,5%
TRF	284	17,4%
PUR	265	17,2%
DPU	145	14,4%
UIR	579	8,8%

*Приблизително структурно измерване на завършващите блокове, не класификация на всеки ред. В някои блокове има и разделители или последващи уточнения.

Виж например `TRF-001`. След изпълнимите изисквания следва completion record с комити, резултати от тестове, първи неуспешен одиторски пас, корекция, втори успешен пас и QA.

012_PLAN_TRF_2026-10-04.md

Това е полезна информация за историята на тикета.

Но за Writer по `TRF-003` тя в повечето случаи е ненужна. Ако има зависимост, той се нуждае от **финалното състояние и конкретния интерфейс**, а не от начина, по който TRF-001 е достигнал до него.

Проблем Б: readiness находките остават смесени с окончателните изисквания

Това е по-сложният проблем.

При `PLAN_PUR` открих редове, които едновременно съдържат:

- окончателно техническо изискване;
- обяснение защо то съществува;
- номер на блокер от по-стар readiness пас;
- описание на ревизия;
- съпоставка с предишен вариант на плана.

Например `PUR-001` описва ограничения на базата, RLS и негативни тестове, но включва и обяснения от блокери на първоначалния PUR v1.

Тук не трябва просто да изтрием всичко, в което пише „блокер“. Част от тези редове са същинските изисквания, родени от одита.

Правилната трансформация е:

Преди — смесен запис

„Блокер 3 от пас 2: необходимо е проверката за фирмата да бъде в транзакцията, защото…“

След — два независими артефакта

Одобрен договор за изпълнение: Проверката за фирмата се извършва в същата транзакция като записа. Acceptance: негативен тест с чужд идентификатор.

Readiness история: Пас 2, блокер 3, причина, доказателство, ревизия и начин на затваряне.

Агентът, който пише кода, получава първото. Агентът, който разследва произхода на решение, може да отвори второто.

Проблем В: старите фази стоят в активния план

Тук PLAN_UIR е най-показателен.

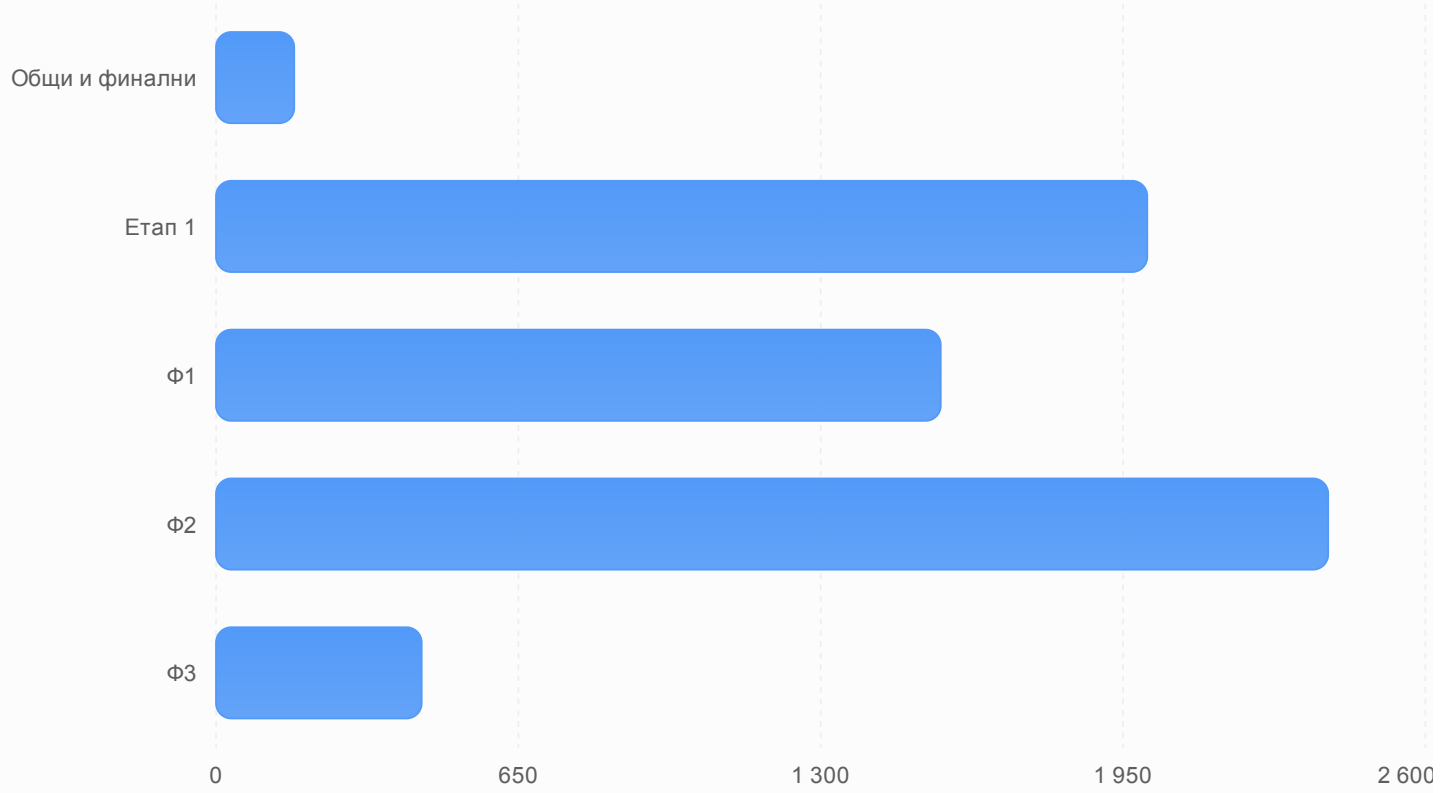
3. PLAN_UIR — основният източник на натрупване

Разделих активния план по реалните му секции.

Секция	Редове	Дял от файла
Въведение, контекст, цели и решения	138	2,1%

Етап 1	2 002	30,5%
Етап 2 — Ф1	1 558	23,7%
Етап 2 — Ф2	2 391	36,4%
Етап 2 — Ф3	442	6,7%
Финални секции	30	0,5%

Процентите са по брой редове; леките отклонения са от закръгляне.



В каченото състояние на PLAN_UIR вече е записано, че Ф1 е затворена на 8 октомври, Ф2 — на 12 октомври, а Ф3 е одобрена за изпълнение на 14 октомври.

PLAN_UIR.md

PLAN_UIR.md

Значи **над 90% от редовете на активния план се намират извън текущата Ф3**.

Това не означава, че 90% от информацията е ненужна. Ф3 наследява правила от Ф1 и Ф2. Но означава, че имаш много голяма възможност да изградиш по-малък *working set* — наборът от документи, който COO и агентите действително трябва да прочетат.

Най-важното ми откритие е във Ф3.1

Там пише приблизително следното: за Ф3 продължават да важат Ф2.1, Ф2.2, Ф2.3, Ф2.4, Ф2.10, Ф2.12, а част от тях препращат към Ф1.

PLAN_UIR.md

Това представлява **верига от наследявани правила между големи текстови секции**.

И тук е архитектурният проблем.

За да бъде сигурен, че разбира Ф3, СОО може да се почувства длъжен да отвори Ф2, после Ф1, после старите решения и ревизии.

Всяко ниво потенциално води до още четене.

Моята препоръка е наследяването да остане, но да стане експлицитно и проверимо, вместо да зависи от препратки между дълги документи.

4. Как точно са го решили конкурентите

Тук GSD се оказва най-полезният образец, но BMad също има няколко много добри решения.

GSD Core — разделя информацията според жизнения ѝ цикъл

Има различни файлове за решенията, изследването, изпълнимите планове, историята на обсъжданията, резултатите от изпълнението и проверките.

BMad Method — разделя по продуктови артефакти и тикети

Архитектурните решения, спецификациите, тикетите, плановете за изпълнение и текущият статус имат отделни места.

Gas Town — използва структурирани работни единици

Задачите, зависимостите и оперативните състояния се съхраняват като Beads, вместо да се управляват само чрез един голям Markdown план.

GSD: най-интересният модел за PNP

GSD има структура от този тип:

Обикновен текст

```
.planning/
  PROJECT.md
  REQUIREMENTS.md
  ROADMAP.md
  STATE.md

  phases/
    03-feature/
```

```
03-CONTEXT.md
03-DISCUSSION-LOG.md
03-RESEARCH.md
03-VALIDATION.md
03-01-PLAN.md
03-01-SUMMARY.md
03-02-PLAN.md
03-02-SUMMARY.md
03-VERIFICATION.md
```

Това е реалната документирана структура, опростена до основните артефакти.

GitHub

Тук има три особено интересни решения.

Първо — `DISCUSSION-LOG.md` не се чете от автоматичните workflow-и. Това е изрично записано в документацията. Той е за историческа проверка, не за изпълнение.

GitHub

Това отговаря почти буквално на въпроса ти за readiness блокерите.

Второ — `PLAN.md` е чистият договор за изпълнение. Отделният `SUMMARY.md` съдържа какво действително е направено, отклоненията и резултата.

Тоест completion record не увеличава изпълнимия план.

Трето — старите решения се обобщават в `DECISIONS-INDEX.md`. Вместо непрекъснато да се четат няколко предишни фази, системата поддържа ограничено по размер резюме на решенията.

GitHub

Има дори изрично правило: предишни `SUMMARY.md` файлове се включват в контекста на нов план само при действителна зависимост, а не автоматично.

GitHub

Точно този принцип бих взел за PNP.

BMad: отделни тикети и малко общо състояние

В актуалния BMad `bmad-build` обработва по една цел или един тикет. За по-голяма инициатива има дърво от тикети, като отделният тикет има собствен план.

Проследяването на състоянието е в `sprint-status.yaml`. Страничните задачи и несвързаните открития се записват в `deferred-work.md`, вместо да се добавят към текущото изпълнение.

BMad Method +1

Интересно е, че генерирането, валидирането и актуализирането на `sprint-status.yaml` се управляват от детерминистичен Python скрипт. AI не трябва да отгатва кой тикет е следващият от разказ в края на дълъг документ.

Това е полезен образец за `PLAN_UIR.md` като индекс и за `checkpoint.md`.

Gas Town: от текстови планове към граф от задачи

Gas Town работи със структурирани Beads: идентификатори, статуси, зависимости, родители, отговорници и други полета. Командата `bd ready` определя кои задачи могат да бъдат започнати.

GitHub +1

Това елиминира нуждата голям Markdown файл да бъде едновременно списък със задачи и механизъм за управление на състоянието.

Но не бих пренасял директно Beads в PNP. Това добавя съхранение, синхронизация и нови режими на отказ. За твоите нужди същият принцип може да се реализира много по-просто с файлове и малък manifest.

Изводът от конкурентите: общият принцип е *bounded context* — всяка задача има ограничен набор от входове. GSD го прилага най-ясно на ниво документи, BMad — на ниво продуктови артефакти и тикети, Gas Town — на ниво структурирано оперативно състояние.

---

5. Как бих проектирал новата структура на PNP

Бих запазил Markdown като основен формат. Няма нужда от база данни, сложен orchestration service или пренаписване на цялата система.

Ето пример как бих структурирал PLAN_UIR:

Предложена структура на мисията

```
dev/backlogs/active/UIR/
├── PLAN_UIR.md          # индекс
├── DECISIONS.md         # одобрени решения
├── STATE.yaml           # фаза и тикет
│
├── phases/
│   ├── F1/
│   │   └── CONTRACT.md
│   ├── F2/
│   │   └── CONTRACT.md
│   └── F3/
│       └── CONTRACT.md
│
├── tickets/
│   ├── UIR-049.md
```

```
│   ├── UIR-050.md
│   └── UIR-051.md
│
└── records/
    ├── UIR-038.md
    ├── UIR-049.md
    └── ...
```

Readiness доказателствата остават в съществуващото `dev/backlogs/readiness/UIR/`. Не ги дублираме в новата структура.

Ролите на файловете

Файл	Какво съдържа	Кой го чете
`PLAN_UIR.md`	Цел, активна фаза, ред на работа, карта на файловете	COO при старт
`DECISIONS.md`	Само действащите одобрени решения и приложимостта им	Planner, COO, Writer по обхват
`phases/F3/CONTRACT.md`	Общи правила и зависимости на Ф3	COO, Writers от Ф3
`tickets/UIR-049.md`	Scope, Outcome, Acceptance, Risk, Stop, Verify	Writer, COO, reviewer
`records/UIR-049.md`	Какво е направено, тестове, одити, комити, отклонения	COO при затваряне, бъдещи зависими тикети по нужда
`readiness/UIR/...`	Сканирания, блокери, ревизии и решения	Planner и readiness одитор
`STATE.yaml`	Текущо състояние, статуси, указатели	Оркестрационният механизъм

А какво става с тежките правила?

Например Ф3 в момента наследява от Ф2 правилата за правата, tenant join-овете, числата по валута, проверките за `own` и тестовите базови линии.

Не бих копирал целия Ф2 текст във Ф3.

Бих направил стабилни, идентифицирани правила в `DECISIONS.md` или в съответния проектен архитектурен канон.

Пример:

YAML

```yaml
ticket: UIR-049
phase: F3
status: approved

depends_on:
  - TRF-002
```

```
  - UIR-021

required_context:
  - phases/F3/CONTRACT.md
  - tickets/UIR-049.md

canonical_rules:
  - TENANCY-01
  - PERMISSIONS-OWN-02
  - MONEY-CURRENCY-01
  - TEST-E2E-04
```

Това е предложена схема, не формат от текущия PNP.

Важното е, че четирите правила се разрешават към **единствените им авторитетни дефиниции**. Не ги преписваме в още пет тикета.

Ако такова правило се промени, системата трябва да знае кои фази и тикети зависят от него.

6. Най-важното: кой агент какво чете

Това според мен трябва да бъде техническо правило на PNP, а не препоръка в промпта.

Роля или етап	Задължителен контекст	По заявка
Readiness COO	Целият предложен обхват, канонични правила, зависимости	Исторически решения и discovery
Readiness одитор	Одитираната версия на всички релевантни планове и правила	Сурови сканове и предишни присъди
Execution COO	Индекс, текуща фаза, активен тикет, checkpoint	Предишни records
Writer	Тикет, релевантни канонични правила, зависимости, текущ код	Discovery и исторически записи
Sonnet fact-check	Технически твърдения от брифа, код и зависимости	Допълнителни доказателства
Codex code auditor	Одитиран diff, договор на тикета, релевантни правила, доказателства	Предишен пас при resume
Closeout	Records и статус на мисията	Останалата история

Тук има едно съществено условие.

Readiness одиторът не бива да получава орязан контекст, който скрива важни зависимости. При одобряване на план със седем взаимозависими тикета той може да има нужда от всички седем и от общите им правила.

Оптимизацията е предимно при изпълнението и възстановяването на сесиите. За readiness целта е правилно подбран, пълен одитен пакет, а не произволно намаляване на входа.

И още нещо: разпределението в малки файлове само по себе си не пречи на агентите да ги прочетат всичките. **Истинската оптимизация е правилото кои файлове се зареждат, подкрепено от проверка на съставения контекст.**

---

7. Колко контекст потенциално можеш да спестиш?

Тук трябва да бъдем внимателни с обещанията.

PLAN_UIR е около 883 хил. символа.

Ф3 заема само около 64 хил. символа, а началната обща част е около 27 хил. Това е приблизително 10% от текущия размер, преди да добавим действително необходимите наследени правила и зависимости.

Целият PLAN_UIR	Ф3 + начална част
883 KB	**~91 KB**
Текущ монолит	Преди допълнителни зависимости

Това показва потенциал за **почти 90% по-малко прочетен текст от самия план**, ако текущата операция действително изисква само тези секции и преди това е четяла целия файл.

Но няма да твърдя, че Claude usage автоматично ще падне с 90%.

Защото:

- Не сме доказали, че всички Writers в PNP четат целия PLAN_UIR. Възможно е COO вече да им изпраща само бриф.
- Началният контекст включва и доктрината, tools, memory, checkpoint и други документи.
- Големите diff review операции също запълват контекста, независимо от размера на плана.
- Prompt cache може да направи повторното четене относително евтино, макар че големият контекст продължава да има цена и оперативни последствия.

Следователно това е възможност за оптимизация, не измерена икономия.

В експеримента с UIR-038 наблюдавахме, че дори след compact COO може да натрупа голям контекст от прегледа на код и подготовката на одита. Разделянето на плановете ще реши само една част от този проблем.

8. Три риска при разделянето, които не бива да допускаме

Риск 1: изгубени зависимости

При TRF, PUR и DPU има реални междупланови зависимости. PUR използва модела на разделените трансфери от TRF; DPU използва резултатите от PUR. Това не са просто три независими файла.

013_PLAN_PUR_2026-10-04.md

014_PLAN_DPU_2026-10-04.md

Затова трябва да има dependency manifest, а не само директория с Markdown файлове.

Риск 2: одитираното да е различно от изпълненото

Това е особено важно за PNP.

Ако Codex одобри тикет при определена версия на фазовите правила, а след това някой промени `DECISIONS.md`, Writer може да изпълнява различен договор от одитирания.

Затова бих въвел **одитен manifest с хешове на всички задължителни файлове**.

При промяна на приложимите изисквания след одобрението — стоп и необходимият нов readiness контрол.

Това е по-силно от просто запис „одобрен“ в началото на един документ.

Риск 3: прекалено раздробяване

Не бих разделял всеки Acceptance, Risk и Verify на отделен файл.

Напротив, целият договор за изпълнение на един тикет трябва да остане на едно място.

Иначе Writer ще трябва да прави десетки четения, ще се появят повече счупени препратки, а поддръжката ще стане по-сложна.

Един тикет = един изпълним договор. Един тикет = един отделен completion record.

При изключително голям тикет вече може да се обсъжда допълнително разделяне, но това не бива да е правило по подразбиране.

---

9. Как бих пренесъл промяната в PNP

Не бих започнал с механично разцепване на всичките шест документа.

Бих го оформил като отделен PNP архитектурен кандидат и бих провел миграция в четири контролирани етапа:

1. **Нов формат, без промяна на съществуващите правила.** Определят се индекс, фазов договор, тикет, completion record, readiness история и manifest. Добавят се структурни тестове.

2. **Миграция върху копие на затворен план.** Например PLAN_TRF. Сравняват се оригиналният и новият изпълним договор, проверяват се всички решения, Acceptance, Risk и зависимости. Оригиналът остава непокътнат като архив.
3. **Проверка на пълнотата.** PNP трябва да установява липсващи тикети, изгубени решения, счупени препратки, невалидни зависимости и несъответствие между одитираната и изпълняваната версия.
4. **Dogfood тест върху следваща фаза на Silerax или Furnissimo.** Сравняват се startup context, общо прочетен текст, cache read/write, Claude usage, време и установени пропуски. Едва след успешен тест новият формат става доктрина.

Не бих мигрирал активния PLAN_UIR насред изпълнението на тикет. При него има твърде много препратки към секции и одобрени зависимости. Бих използвал следваща фазова граница или нов план като момент за реалното превключване.

10. Как да докажем дали оптимизацията си заслужава

Имаме данни за общия контекст на COO, но нямаме измерване кои точно части от плановете чете всеки агент.

Това е пропускът, който трябва да запълним.

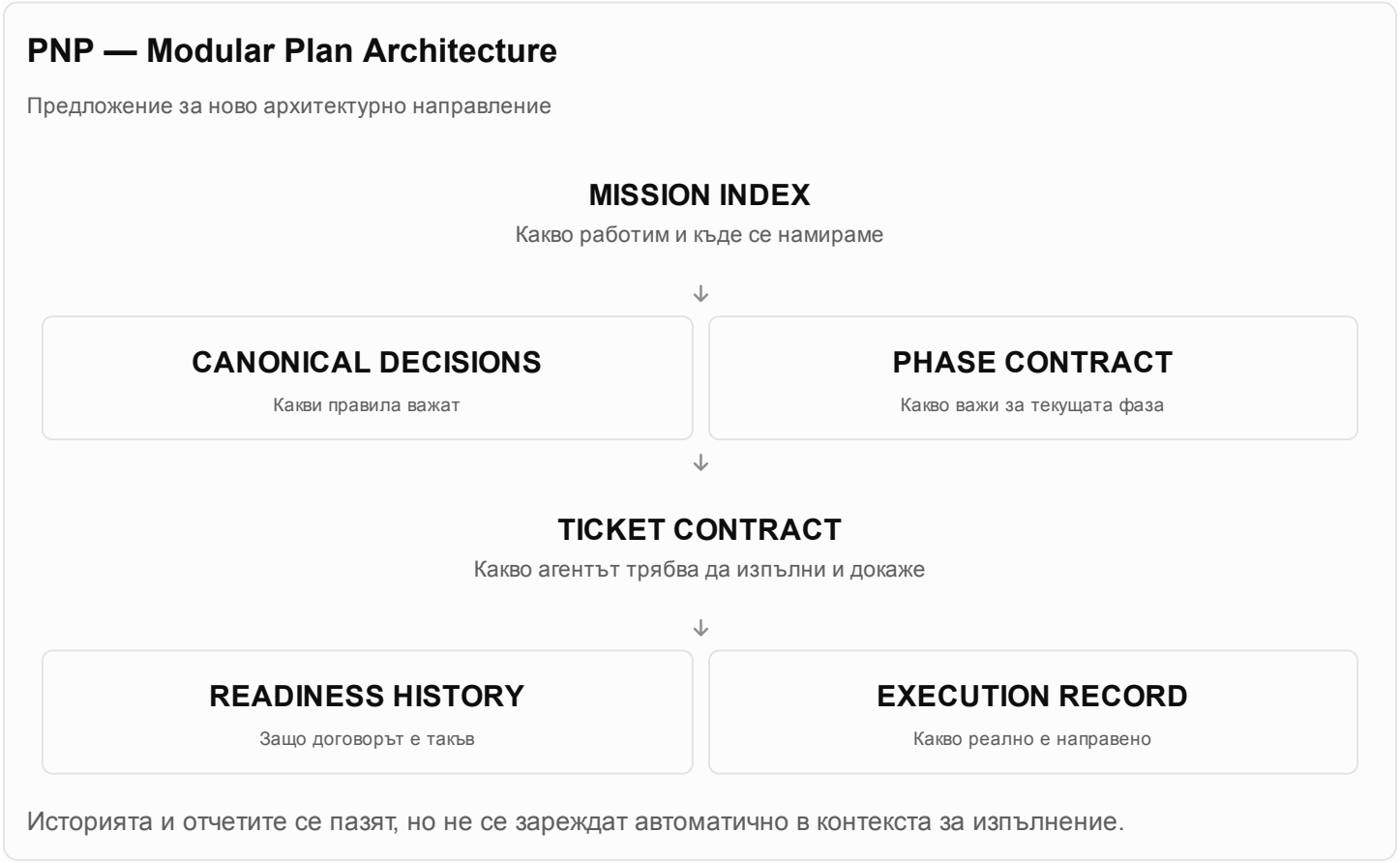
Бих направил кратък експеримент с две конфигурации на една и съща задача:

Метрика	Монолитен план	Разделен план
Начален контекст на COO	Измерва се	Измерва се
Прочетени символи от плановете	Измерва се	Измерва се
Контекст на Writer	Измерва се	Измерва се
Cache read/write	Измерва се	Измерва се
Общо Claude usage	Измерва се	Измерва се
Време до завършен тикет	Измерва се	Измерва се
Изгубени изисквания	Проверка	Проверка
Блокери на одитора	Отчитат се	Отчитат се

Само равен обхват, сходни модели и еднакви проверки биха направили разликата смислена. Иначе може да оптимизираме контекста на хартия, но да увеличим повторните четения и одиторските рундове.

Окончателната ми препоръка

След тези шест плана бих предложил на PNP следната архитектурна промяна:



Това според мен ще позволи на PNP да запази своята дисциплина и проследимост, като същевременно стане по-лек при изпълнение.

Какво бих взел от конкурентите? От GSD — организацията на артефактите и селективното четене. От BMad — един изпълним договор за една задача и отделно управление на статуса. От Gas Town — идеята за явен граф на зависимостите, но без тежката му инфраструктура.

Какво бих запазил от PNP? Независимия одит, операторските гейтове, техническите защити, изискването за доказателства, личния COO преглед и ledger-а. Те не са причината плановете да се раздуват; причината е начинът, по който се съхранява резултатът от тези процеси.

И най-важното: **не бих оптимизирал PNP просто да чете по-малко. Бих го оптимизирал никога да не пропуска важно изискване, докато чете възможно най-малко.**

Това е правилната цел, особено след като вече видяхме колко струва един излишен Codex пас.

Според мен тази промяна е достатъчно важна, за да стане самостоятелна архитектурна мисия в PNP, а не поредното малко правило в `PROJECT_OVERRIDES.md`.